System programming for Systems on Chip Practical Work 2

Hsing-Yu Lin(407209), Sheng-Wen Chen(414932)

October 5, 2025

1 Fixed point Mandelbrot

1.1 Why Q4.28 and how to handle overflow

We slightly revised the algorithm, which remains functionally equivalent to the original. As it can be seen from below, instead of using a do-while loop, we used a while loop. This is for the sake of adding the `is_escaped_q4_28(`x_{i+1}, y_{i+1}`)` function, which is explained below.

```
uint16_t calc_mandelbrot_point_soft(fxpt_4_28 cx, fxpt_4_28 cy, uint16_t n_max)
{
  fxpt_4_28 x = cx;
  fxpt_4_28 y = cy;
  uint16_t n = 1;
  fxpt_4_28 xx, yy, two_xy;
  while ((!is_escaped_q4_28(x,y)) && (n < n_max))
  {
    xx = fxpt_4_28_mul(x, x);
    yy = fxpt_4_28_mul(y, y);
    two_xy = fxpt_4_28_mul(x, y) << 1;
    x = xx - yy + cx;
    y = two_xy + cy;
    ++n;
  }
  return n;
}
```

Figure 1: calc_mandelbrot_point_soft function with fixed-point

In order to find the suitable position to put the decimal point, we firstly need to calculate the range of the computation values, Given `SCREEN_WIDTH` = 512, `SCREEN_HEIGHT` = 512, `FRAC_WIDTH` = 3.0, $c_x^{(0)} = -2.0$, $c_y^{(0)} = -1.5$. If x_i, y_i satisfy $x_i^2 + y_i^2 < 4.0$, then:

1. range of x_i^2 :	$0 \le x_i^2 < 4.0$
2. range of y_i^2 :	$0 \le y_i^2 < 4.0$
3. range of $2x_iy_i$:	$-4.0 < 2x_iy_i < 4.0$
4. range of $(x_i^2 - y_i^2)$:	$-4.0 < x_i^2 - y_i^2 < 4.0$
5. range of c_x :	$-2.0 \le C_x < 1.0$
6. range of (x_{i+1}) :	$-6.0 < x_{i+1} < 5.0$
7. range of c_y :	$-1.5 \le C_y < 1.5$
8. range of (y_{i+1}) :	$-5.5 < y_{i+1} < 5.5$

Growth estimate of Z.

$$|Z_{\mathrm{i}+1}| = \left|Z_{\mathrm{i}}^2 + C\right| \;\le\; |Z_{\mathrm{i}}|^2 + |C| \;\le\; x_{\mathrm{i}}^2 + y_{\mathrm{i}}^2 + |C| \;<\; 4 + |C|\,,$$

with $C = (C_x, C_y)$, where $C_x \in [-2, 1)$ and $C_y \in [-1.5, 1.5)$. Hence

$$|C| \;\le\; \sqrt{(-2)^2 + (1.5)^2} \;=\; 2.5, \qquad |Z_{\mathrm{i}+1}| \;<\; 4 + |C| \;\le\; 6.5,$$

and therefore

$$x_{\mathrm{i}+1}^2 + y_{\mathrm{i}+1}^2 \;=\; |Z_{\mathrm{i}+1}|^2 \;<\; 6.5^2 \;=\; 42.25$$

Only $x_{i+1}^2 + y_{i+1}^2$ can exceed 8. Thus, to maximize precision, we adopted a signed Q4.28 fixed-point format with range $(-8, 8)$, and use `is_escaped_q4_28`(x_{i+1}, y_{i+1}) to detect escapes and prevent overflow whenever $x_{i+1}^2 + y_{i+1}^2 > 8$. With Q4.28, bit 31 is a sign bit, bit 30, 29 and 28 (3 bits) are used to represent the integer part of the value, which is up to 7. The remaining bits are then used to represent the fraction part of the value, thus forming a fixed-point representation that has a range of (-8, 8).

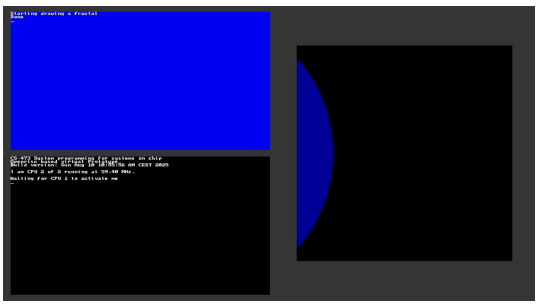
```
uint16_t is_escaped_q4_28(fxpt_4_28 x, fxpt_4_28 y)
{

    static const fxpt_4_28 TWO_4_28 = FTOFIX(2.0);
    static const fxpt_4_28 FOUR_4_28 = FTOFIX(4.0);
    if (x >  TWO_4_28 || x < -TWO_4_28 || y >  TWO_4_28 || y < -TWO_4_28)
    {
        return 1;
    }
    fxpt_4_28 result = fxpt_4_28_mul(x, x) + fxpt_4_28_mul(y, y);
    return (result >= FOUR_4_28) ? 1 : 0;
}
```

Figure 2: is_escaped function to check for overflow.

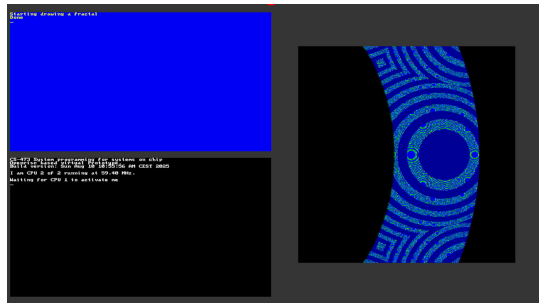
Table 1 shows the difference between using and not using `is_escaped_q4_28`(x_{i+1}, y_{i+1}). The initial values are $C_{x,0} = 1.5$ and $C_{y,0} = -1.5$. With this setting, the image center corresponds to $(C_x, C_y) = (3, 0)$ (the delta is 3/512, meaning that (C_x, C_y) at the center of the image will be $(C_x, C_y) = (1.5 + 1, 5, -1.5 + 1.5)$). For any point $p = (x, y)$ inside the open disk $B((3, 0), 1)$ we have

$$\|p\| \;\ge\; \|(3,0)\| - \|p - (3,0)\| \;>\; 3 - 1 \;=\; 2,$$

In other words, **for any point p inside the open disk centered at (3,0) with radius 1, the distance from p to the origin is always strictly greater than 2**. Hence $x^2 + y^2 = \|p\|^2 > 4$.



With `is_escaped_q4_28()`



Without `is_escaped_q4_28()`

Table 1: With and without `is_escaped_q4_28()` for $c_x(0) = 1.5$, $c_y(0) = -1.5$.

Therefore, the entire middle band is expected to be *uniformly black*. Without the `is_escaped_q4_28` guard, the Q4.28 squaring and accumulation operations can cause *integer overflow* (two's-complement wrap-around) near the boundary, leading to spurious rings/stripes. Enabling `is_escaped_q4_28` prevents these overflows and restores the correct solid-black region.

1.2 Data type conversion

To convert a float to our fixed-point representation, we multiply f by 268,435,456.0f (that is, 2^{28}, equivalent to a 28-bit left shift) and cast to our fixed type (a signed 32-bit integer). Since we have already shifted the decimal portion up with the multiplication, we don't actually lose the decimal portion during the cast (Figure 3). The conversion from fixed-point to float is done by dividing `x` by 268,435,456.0f (that is, 2^{28}, equivalent to a 28-bit right shift) and set it back into the IEEE 754 representation by casting (Figure 4).

```
#define FTOFIX(f)      ((fxpt_4_28)((f) * 268435456.0f))
```

Figure 3: Floating-point to fixed-point conversion.

```
#define FIXTOF(x)      ((float)((x) / 268435456.0f))
```

Figure 4: Fixed-point to floating-point conversion.

To make it easier to understand, say we want to convert the float 3.0 to the fixed-point representation. With our conversion method we would first do 3 * 2^{28} and cast to an integer type to get the Q4.28 representation (in binary it is represented as 00110000000000000000000000000000). Afterwards, if we want to convert it back, we will just divide 3.0 * 2^{28} by 2^{28} to make it 3.0 again (after the cast to float, 3.0 will be represented back as the IEEE754 again). The arithmetic operations with this method are relatively simple. An example of the arithmetic operations will be shown in the next section.

1.3 Fixed point arithmetic operations

For addition and subtraction we treat the operands like ordinary 32-bit integers, because both values share the same fixed scale 2^{28}; the binary point is already aligned, so no rescaling is needed (aside from optional saturation).

For multiplication, the fractional length doubles, so the raw product is Q8.56. Since each Q4.28 operand is scaled by 2^{28}, their product is scaled by 2^{56}. To store the result as Q4.28, we compute the product in 64 bits, then right-shift by 28 bits to keep 28 fractional bits, and finally cast to the 32-bit Q4.28 fixed-point format (Figure 5).

For example, say we want to do the calculation of float 3.0 * float 2.0. In fixed-point arithmetic, this will be done as (3 * 2^{28}) * (2 * 2^{28}) which is 3 * 2 * 2^{56}. Since we want 6 * 2^{28}, we will right-shift 3 * 2 * 2^{56} by 28 to obtain the desired value.

```
// Fixed Point: Multiplication
fixed mul_fixed_point(fixed a, fixed b) {
  // a * b is same as
  // a * 2^f * b * 2^f = (a * b) * 2^f
  // meaning that a * b has the binary point at bit '2f'
  // to keep the binary point at f, you need to rescale by 2^f
  // that is a * b / 2^f or a * b >> f

  // This operation will throw away f fractional bits
  // Thus rounding may be needed
  int64_t fixed_2q_2f = (int64_t)a * (int64_t)b;
  fixed mul_num = (fixed)(fixed_2q_2f >> FIXED_POINT_FRAC_BITS);
  return mul_num;
}
```

Figure 5: Fixed point multiplication

2 Our own floating point format

2.1 Data type conversion

Our custom floating-point format is designed with a limited range and a unique exponent bias rule. The total representation uses 32 bits, allocated as follows:

- **Sign Bit (1 bit):** S (0 for positive, 1 for negative).
- **Exponent (2 bits):** E.
- **Mantissa (29 bits):** M.

This setup sets the representable range of the value to $(-8, 8)$.

1. **Exponent Range:** Our exponent bias is set to **0**. The 2-bit exponent field, E, equals the true exponent $e \in \{0, 1, 2, 3\}$. This corresponds to a multiplier of $2^e \in \{2^0, 2^1, 2^2, 2^3\}$.
2. **Mantissa Range:** Unlike the standard IEEE 754 format, we do not use an implicit leading 1 in the significand (no hidden bit). The mantissa M has a range of $\mathbf{0.0} \leq \mathbf{M} < \mathbf{1.0}$ (i.e., it represents the fractional part only).

The actual value V is calculated using the formula:

$$V = (-1)^S \times M \times 2^e$$

To illustrate, the number **3.0** is represented as **0 10 11000...0**.

- **Sign Bit (S):** 0 (positive number).
- **Exponent Bits (E):** 10, meaning the exponent $e = 2$, and the multiplier is $2^2 = 4$.
- **Mantissa (M):** The binary fraction $0.11\ldots$ is interpreted as $0.5 + 0.25 = 0.75$.

The final value is: $V = 1 \times 0.75 \times 2^2 = 3.0$.

Comparing this to the IEEE 754 standard, which would represent 3.0 using 1.5×2^1, we see a fundamental difference in how the exponent is determined, since our mantissa does not include the

	signed/ unsigned	exponent	mantissa
3.0	0	10	1100 0000 0000 0000 0000 000
2.0	0	10	1000 0000 0000 0000 0000 000
1.5	0	01	1100 0000 0000 0000 0000 000
1.0	0	01	1000 0000 0000 0000 0000 000
0.5	0	00	1000 0000 0000 0000 0000 000
0.0	0	00	0000 0000 0000 0000 0000 000
-0.5	1	00	1000 0000 0000 0000 0000 000
-1.0	1	01	1000 0000 0000 0000 0000 000
-1.5	1	10	1100 0000 0000 0000 0000 000
-2.0	1	10	1000 0000 0000 0000 0000 000
-3.0	1	10	1100 0000 0000 0000 0000 000

Figure 6: Range of values from -3.0 to 3.0 in our floating-point representation.

leading '1'. The representation of positive and negative numbers from -3.0 to 3.0 is visualized in Figure 6.

Figure 7 outlines the conversion function. The core idea is to normalize the floating-point number F by dividing it by 2 until it is less than 1.0. The number of divisions performed determines the exponent bits, and the resulting fractional part becomes the mantissa.

2.2 Conversion Steps for 3.0

To convert **3.0** to the final representation **0 10 11000**...**0**:

1. **Signed Bit (S):** Since 3.0 is positive, the signed bit is **0**.
2. **Exponent Calculation:** We repeatedly divide 3.0 by 2 until the result is less than 1.0.
   - $3.0/2 = 1.5$ (Division count: 1)
   - $1.5/2 = 0.75$ (Division count: 2)

   The final division count of 2 corresponds to the exponent bits **10**.
3. **Mantissa Calculation (M):** The fractional part 0.75 is converted into the 29-bit integer mantissa field. This is achieved by multiplying the fraction by 2^{29} and casting the result to an unsigned integer, similar to fixed-point conversion:

   $$M_{\text{int}} = \lfloor 0.75 \times 2^{29} \rfloor$$

   This integer value represents the binary fraction 0.11000...0.
4. **Final Combination:** The final three components are then combined using a bitwise OR operation via the 'myft_pack()' function to form the 32-bit floating-point representation:

Signed Component:	0	00 000...0
Exponent Component:	0	10 000...0
Mantissa Component:	0	00 110...0
Final Representation:	0	10 110...0

Figure 8 displays a better comparison between our 3.0 representation and the IEEE754.

2.3 Floating-Point Arithmetic Operations

The floating-point arithmetic operation in our implementation is performed using standard methods: the **exponents are added** together, and the **mantissa parts are multiplied** together (Figure 9).

Similar to the fixed-point representation, the multiplication of two 29-bit mantissas yields a 58-bit product. Therefore, a **right shift by 29 bits** is required after multiplication to normalize the result.

The sum of two addends' exponents may **overflow** (exceed the capacity of the 2-bit exponent field). To handle this, a `while` loop checks whether the overflowed sum is shiftable (i.e., whether the resulting number is still within the valid range of the format).

Example:

- Float 2.0 in our custom representation is: 0 10 **1**000000...000 (representing 0.5×2^2).
- When calculating 2.0×2.0, the exponent bits 10 and 10 are added, resulting in an exponent bits of 100 (2^4). This value, when encoded, will exceed the 2-bit capacity.
- The product is conceptually $0.25 \times 2^4 = 4.0$.
- Since 4.0 is still within our floating-point range, we perform a denormalization shift to bring the exponent back into the 2-bit range:

  $$0.25 \times 2^4 = (0.25 \times 2) \times (2^4/2) = 0.5 \times 2^3$$

  The exponent is shifted right, and the mantissa is shifted left to compensate. The product $2.0 \times 2.0 = 4.0$ can then be correctly represented as 0 11 10000000000000000000000000000

```c
// Data type conversion
// converts from IEEE754 to our data tupe format
// signed_bit(bit 31)    exponent_bit(bit 30, 29)    mantissa_bit(bit 28~0)
//         0                        00                      0000...0000
myft float_to_myft(float f)
{
    if (f == 0.0f)
    {
        return 0;
    }

    uint32_t sign = (f < 0.0f) ? 1u : 0u;
    float x = (f > 0) ? f : -f;
    uint32_t exp = 0;

    // x / 2 division is done until 3 times (exponent reaches 3)
    // as we have 2 bits for the exponent, decimal(11)=3 is the largest exponent we can have
    while (exp < 3 && x >= 1.0f)
    {
        x *= 0.5f;
        exp++;
    }

    // the remainder of x should be the mantissa part
    /**Ex)
        * say the f = 3.0
        * after the while loop
        * x == 0.75
        * exp == 2
     */
    // We want to convert 0.75 to 11000...
    // to do this, we will use the idea of the fixed-point representation
    // and do x * (2^29), making the float to an integer
    double md = (double)x * (double) MANTISSA_ONE;
    uint64_t mantissa = (uint64_t)md;

    return myft_pack(sign,exp, (uint32_t) mantissa);
}
```

Figure 7: Flowchart of the conversion function from standard float to our custom 32-bit floating-point format.

bits	31		30	29	28	27	26	25	24	23		22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
												0.5	0.25	0.125	0.0625																			
3.0 IEEE 754	0		1	0	0	0	0	0	0	0		1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

bits	31		30	29		28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
						0.5	0.25	0.125	0.0625																									
3.0 myflpt	0		1	0		1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 8: 3.0 from IEEE754 and our floating-point representation

```c
myft myft_mul(myft lhs, myft rhs)
    // using XOR operation for the signed bit
    uint32_t prod_sign = myft_sign(lhs) ^ myft_sign(rhs);

    // exponent can be added directly
    // lhs = A * 2^a
    // rhs - B * 2^b
    // lhs * rhs = A * B * 2^(a + b)
    // where A and B are mantissa
    uint32_t prod_exp = myft_exp(lhs) + myft_exp(rhs);

    // since A and B mantissa are 29 bits
    // 29 bits * 29 bits can be 58 bits
    // thus we need to shift >> 29 bits to make it 29 bits
    // (some very small values will be lost during this process)
    uint64_t lhs_mantissa = (uint64_t)myft_mantissa(lhs);
    uint64_t rhs_mantissa = (uint64_t)myft_mantissa(rhs);
    uint32_t prod_mantissa = (lhs_mantissa * rhs_mantissa) >> 29;

    if(prod_mantissa == 0)
    {
        return 0;
    }

    // dealing with overflow of exponent addition
    /**Ex) say that we have 2.0 * 2.0
     *     where in our floating point representation
     *     2.0 = 0 10 10000000000000000000000000000
     *
     *     mantissa multiplication will be
     *        10000000000000000000000000000
     *      * 10000000000000000000000000000
     *      -------------------------------
     *        01000000000000000000000000000000 (after shift)
     *
     *     the expoent is 10 so, 10 + 10 = 100, which exceeds the
     *     2 bit capacity of the exponent. Here the calculated result is
     *     representing (0.25 * 2^4)
     *
     *     To handle this we shift right the exponent and shift left the mantissa.
     *     In simple terms (0.25 * 2) * (2^4 / 2) = 0.5 * 2^3
     *
     *     exponent: 100 --> 11
     *     0 11 10000000000000000000000000000
     *
     *     forming
     *     4.0 = 0 11 10000000000000000000000000000
     */
    while (prod_exp > 3u)
    {
        // check whether the production of the mantissa is still shiftable
        // if for instance prod_mantissa = 10000000000000000000000000000
        // it cannot be shifted anymore
        if (prod_mantissa <= (MANTISSA_MASK >> 1))
        {
            prod_mantissa <<= 1; // mantissa * 2, as we -1 the exponent
            prod_exp-- ;
        } else
        {
            // overflow
            prod_exp = 3u;
            prod_mantissa = MANTISSA_MASK;
            break;
        }
    }
    return myft_pack(prod_sign, prod_exp, prod_mantissa);
}
```

Figure 9: myflpt multiplication

```
-3.0  11011000000000000000000000000000
-2.145  11010001001010001111010111000000
-2.0  11010000000000000000000000000000
-1.5  10111000000000000000000000000000
-0.5  10010000000000000000000000000000
0.0  00000000000000000000000000000000
0.5  00010000000000000000000000000000
1.5  00111000000000000000000000000000
2.0  01010000000000000000000000000000
3.0  01011000000000000000000000000000
4.0  01110000000000000000000000000000
6.0  01111000000000000000000000000000
-6.0  11111000000000000000000000000000
3.0 + 3.0 =  01111000000000000000000000000000
-3.0 + (-3.0) =  11111000000000000000000000000000
3.0 + (-1.5) =  01001100000000000000000000000000
-1.5 + (-1.5) =  11011000000000000000000000000000
4.0 + 6.0 =  01111111111111111111111111111111
2.0 - 3.0 =  11001000000000000000000000000000
-3.0 - (-3.0) =  00000000000000000000000000000000
5.0 - (-1.5) =  01111010000000000000000000000000
6.0 - 1.5 =  01110010000000000000000000000000
2.0 * 2.0 =  01110000000000000000000000000000
1.5 * 2.0 =  01101100000000000000000000000000
1.5 * 2.0 =  11101100000000000000000000000000
```

Figure 10: myflpt conversion and arithmetic example

```
    ++n;
} while ( (myft_add(xx, yy) < FOUR_MYFT) && (n < n_max));
```

Figure 11: myflpt while loop no is_escaped() function

One thing to note here is that the floating-point representation after an arithmetic operation will not always be unique for the same value.

- 1.5×2.0 results in 0 11 01100...00 (representing $0.375 \times 2^3 = 3.0$).
- Directly converting the float 3.0 might yield 0 10 11000...00 (representing $0.75 \times 2^2 = 3.0$).

Both binary patterns correctly refer to the value 3.0.

To check this, you can run `./test` in `fractal/myflp/src`. You will see the float conversion example and the calculation example in the terminal (Figure 10).

For the addition function, please check the comments in the `myft_add()` function. Similar to the multiplication function, there is also a `while` loop that checks for overflow and adjusts the exponent and mantissa accordingly.

2.4 Overflow Prevention

Unlike the fixed-point implementation, all of our overflow conditions are handled within the **arithmetic functions** themselves. This is done such that when an overflow occurs, the comparison $x^2 + y^2 < 4$ will automatically fail and break from the Mandelbrot iteration loop (Figure 11).

The `is_escaped_myft()` was initially converted to the floating-point version and used similarly to the fixed-point representation; however, we observed a significant decrease in performance. Thus, we decided to simply handle the overflow directly within the arithmetic operations.

3 Conclusion

Overall, the performance of our fixed-point representation and floating-point representation implementation drastically improved over the built-in floating-point representation. Especially for the Q4.28 fixed point representation, it only took 16 seconds for us to compute the entire Mandelbrot graph; meanwhile, it took the built-in floating point around 1.5 minutes. Our floating point representation was also 10 seconds faster than the built-in floating point representation.